

Name: Ansan Jatak

ENROLLMENT: MITU22BTCS0122

Subject: HPCL

HPC Lab — Day 1 Assignment

Part 1: Linux Commands

1. Directory Creation

Created the required directory structure inside the home folder with a single `mkdir -p` command using brace expansion:

```
mkdir -p Parallel_Programming/{OpenMP,MPI,CUDA,Scripts} find Parallel_Programming | sort
```

```
Parallel_Programming
```

```
Parallel_Programming/CUDA
```

```
Parallel_Programming/MPI
```

```
Parallel_Programming/OpenMP
```

```
Parallel_Programming/Scripts
```

2. File Creation

Created each required source file inside its respective folder: **OpenMP/omp_hello.c**

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {
```

```
    #pragma omp parallel
```

```
    {
```

```
        printf("Hello from thread %d\n", omp_get_thread_num());
```

```
    }    return 0;
```

```
}
```

MPI/mpi_hello.c

```
#include <mpi.h>

#include <stdio.h>

int main(int argc, char** argv) {    MPI_Init(&argc, &argv);

    int rank;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);    printf("Hello from MPI process %d\n", rank);

    MPI_Finalize();

    return 0;

}
```

CUDA/vector_add.cu

```
#include <stdio.h>

__global__ void vectorAdd(int *a, int *b, int *c, int n) {    int i = threadIdx.x;    if (i < n)

        c[i] = a[i] + b[i];

}

int main() {

    printf("Vector addition kernel placeholder\n");    return 0;

}
```

Scripts/compile.sh

```
#!/bin/bash

echo "Compile script placeholder"
```

Verifying all four files were created:

```
find . -type f | sort
```

```
./CUDA/vector_add.cu
./MPI/mpi_hello.c
./OpenMP/omp_hello.c
./Scripts/compile.sh
```

3. Navigation

Using only the keyboard: moved into CUDA, returned home, then moved to Scripts using a relative path.

```
cd Parallel_Programming/CUDA pwd
```

```
/home/user/Parallel_Programming/CUDA
```

```
cd ~ pwd
```

```
/home/user
```

```
cd Parallel_Programming/Scripts pwd
```

```
/home/user/Parallel_Programming/Scripts
```

4. Listing Files

ls -a shows all files including hidden ones; ls -la adds long-format details; ls -lah adds human-readable sizes.

```
cd ~/Parallel_Programming ls -a
```

```
.  ..  CUDA  MPI  OpenMP  Scripts
```

```
ls -la
```

```
total 24 drwxr-xr-x 6 user user 4096 Jul 28 07:21 .
```

```
drwxr-xr-x 3 user user 4096 Jul 28 07:21 .. drwxr-xr-x 2 user user 4096 Jul 28 07:21 CUDA drwxr-xr-x 2 user user 4096  
Jul 28 07:21 MPI drwxr-xr-x 2 user user 4096 Jul 28 07:21 OpenMP drwxr-xr-x 2 user user 4096 Jul 28 07:21 Scripts
```

```
ls -lah OpenMP MPI CUDA Scripts
```

```
OpenMP: total 12K
```

```
-rw-r--r-- 1 user user 167 Jul 28 07:21 omp_hello.c
```

```
MPI:
```

```
total 12K
```

```
-rw-r--r-- 1 user user 240 Jul 28 07:21 mpi_hello.c
```

```
CUDA:
```

```
total 12K
```

```
-rw-r--r-- 1 user user 231 Jul 28 07:21 vector_add.cu
```

```
Scripts: total 12K
```

```
-rw-r--r-- 1 user user 46 Jul 28 07:21 compile.sh
```

5. Copying Files

```
cp OpenMP/omp_hello.c CUDA/ ls CUDA/
```

```
omp_hello.c  vector_add.cu
```

6. Moving Files

```
mv CUDA/vector_add.cu OpenMP/ ls CUDA/ OpenMP/
```

```
CUDA/:
```

```
omp_hello.c
```

```
OpenMP/: omp_hello.c vector_add.cu
```

7. Renaming Files

```
cd CUDA mv omp_hello.c hello_openmp.c ls
```

```
hello_openmp.c
```

8. Deleting Files

```
rm hello_openmp.c
```

```
ls cd ..
```

```
ls -ld CUDA
```

```
(CUDA directory is now empty)
```

```
drwxr-xr-x 2 user user 4096 Jul 28 07:22 CUDA
```

The file was deleted, but the CUDA directory itself still exists, confirmed by `ls -ld`.

9. Searching

```
find ~/Parallel_Programming -type f -name "*.c"
```

```
/home/user/Parallel_Programming/OpenMP/omp_hello.c
```

```
/home/user/Parallel_Programming/MPI/mpi_hello.c
```

10. Finding Current Location

```
pwd
```

```
/home/user/Parallel_Programming
```

11. Viewing File Contents

Created a 5-line commands.txt and viewed it with all five tools:

```
cat commands.txt
```

```
ls -la cd /home/user mkdir project cp file1.txt file2.txt rm file1.txt
```

more and less page through the same content interactively (less additionally allows scrolling backward and searching within the file):

```
more commands.txt
```

```
ls -la cd /home/user mkdir project cp file1.txt file2.txt rm file1.txt  
(END)
```

```
less commands.txt
```

```
ls -la cd /home/user mkdir project cp file1.txt file2.txt rm file1.txt  
commands.txt (press q to quit)
```

```
head -3 commands.txt
```

```
ls -la cd /home/user mkdir project
```

```
tail -3 commands.txt
```

```
mkdir project cp file1.txt file2.txt
```

```
rm file1.txt
```

12. Searching Text

Created linux_commands.txt containing command names, then searched for chmod:

```
cat linux_commands.txt
```

```
ls cd pwd mkdir rmdir cp mv rm chmod chown find grep tar cat head tail
```

```
grep "chmod" linux_commands.txt
```

```
chmod
```

13. Counting

```
wc -l linux_commands.txt wc -w linux_commands.txt wc -c linux_commands.txt wc linux_commands.txt
```

```
16 linux_commands.txt      (lines)
```

```
16 linux_commands.txt      (words)
```

```
71 linux_commands.txt      (characters)
```

```
16 16 71 linux_commands.txt (lines words chars, combined)
```

14. File Permissions

```
ls -l Scripts/compile.sh chmod +x Scripts/compile.sh ls -l Scripts/compile.sh
```

```
-rw-r--r-- 1 user user 46 Jul 28 07:21 Scripts/compile.sh
```

```
-rwxr-xr-x 1 user user 46 Jul 28 07:21 Scripts/compile.sh
```

The permission bits changed from `rw-r--r--` to `rwxr-xr-x`, confirming execute permission was added for owner, group, and others.

15. Compression

```
tar -czvf parallel.tar.gz Parallel_Programming ls -lh parallel.tar.gz
```

```
Parallel_Programming/
```

```
Parallel_Programming/Scripts/
```

```
Parallel_Programming/Scripts/compile.sh
```

```
Parallel_Programming/CUDA/
```

```
Parallel_Programming/OpenMP/
```

```
Parallel_Programming/OpenMP/vector_add.cu
```

```
Parallel_Programming/OpenMP/omp_hello.c
```

```
Parallel_Programming/MPI/
```

```
Parallel_Programming/MPI/mpi_hello.c
```

```
-rw-r--r-- 1 user user 684 Jul 28 07:22 parallel.tar.gz
```

```
mkdir extracted_parallel
```

```
tar -xzvf parallel.tar.gz -C extracted_parallel find extracted_parallel -type f | sort
```

```
extracted_parallel/Parallel_Programming/MPI/mpi_hello.c extracted_parallel/Parallel_Programming/OpenMP/omp_hello.c  
extracted_parallel/Parallel_Programming/OpenMP/vector_add.cu extracted_parallel/Parallel_Programming/Scripts/compile.sh
```

Part 2: Shell Scripting Practice

16. Hello World Script

```
#!/bin/bash
```

```
echo "Welcome to Parallel Programming"
```

```
chmod +x 16_hello.sh ./16_hello.sh
```

```
Welcome to Parallel Programming
```

17. Variables

Replace the placeholder values below with your actual Name, Roll Number, and Branch before submitting:

```
#!/bin/bash
```

```
Name="Your Name Here"
```

```
RollNumber="Your Roll Number Here"
```

```
Branch="Your Branch Here" echo "Name: $Name"
```

```
echo "Roll Number: $RollNumber" echo "Branch: $Branch"
```

```
./17_variables.sh
```

```
Name: Your Name Here
```

```
Roll Number: Your Roll Number Here
```

```
Branch: Your Branch Here
```

18. User Input

```
#!/bin/bash

read -p "Enter Program Name: " ProgramName echo "Compiling $ProgramName"
```

```
./18_userinput.sh
```

```
Enter Program Name: vector_add.cu Compiling vector_add.cu
```

19. Arithmetic

```
#!/bin/bash

read -p "Enter first number: " a read -p "Enter second number: " b echo "Sum: $((a + b))" echo "Difference: $((a - b))" echo "Product: $((a * b))" echo "Quotient: $((a / b))"
```

```
./19_arithmetic.sh
```

```
Enter first number: 20
```

```
Enter second number: 4
```

```
Sum: 24
```

```
Difference: 16
```

```
Product: 80
```

```
Quotient: 5
```

20. Even/Odd

```
#!/bin/bash
```

```
read -p "Enter a number: " num if [  $\$(num \% 2)$  -eq 0 ]; then    echo "$num is Even" else  
    echo "$num is Odd" fi
```

```
./20_evenodd.sh
```

```
Enter a number: 14
```

```
14 is Even
```

```
Enter a number: 7
```

```
7 is Odd
```

21. Largest Number

```
#!/bin/bash read -p "Enter first number: " a read -p "Enter second number: " b read -p "Enter third number: " c
```

```
if [ $a -ge $b ] && [ $a -ge $c ]; then    echo "Largest number is: $a" elif [ $b -ge $a ] && [ $b -ge $c ];  
then    echo "Largest number is: $b" else  
    echo "Largest number is: $c" fi
```

```
./21_largest.sh
```

```
Enter first number: 12
```

```
Enter second number: 45
```

```
Enter third number: 27
```

```
Largest number is: 45
```

22. For Loop

```
#!/bin/bash for i in $(seq 1 10) do  
  
    echo "GPU Programming" done
```

```
./22_forloop.sh
```

```
GPU Programming  
  
GPU Programming  
  
GPU Programming  
  
GPU Programming  
  
GPU Programming  
  
GPU Programming  
  
GPU Programming
```

GPU Programming

GPU Programming

GPU Programming

23. While Loop

```
#!/bin/bash i=1  
  
while [ $i -le 20 ] do  
  
    echo -n "$i "      i=$((i + 1))  
  
done echo
```

```
./23_whileloop.sh
```

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

24. File Existence

```
#!/bin/bash
```

```
read -p "Enter file name to check: " fname if [ -f "$fname" ]; then    echo "File Found" else  
    echo "File Not Found" fi
```

```
./24_fileexists.sh
```

```
Enter file name to check: 16_hello.sh
```

```
File Found
```

```
Enter file name to check: does_not_exist.txt File Not Found
```

25. Counting Files

```
#!/bin/bash
```

```
count=$(ls -p | grep -v / | wc -l)
```

```
echo "Number of files in current directory: $count"
```

```
./25_countfiles.sh
```

```
Number of files in current directory: 10
```

26. Compilation Script

Compiles omp_hello.c with gcc, checks the exit status (\$?) of the previous command, and reports success/failure. The compiled binary was then actually run with 4 threads to confirm it works:

```
#!/bin/bash
```

```
SOURCE="omp_hello.c"
```

```
OUTPUT="omp_hello"
```

```
gcc -fopenmp "$SOURCE" -o "$OUTPUT"
```

```
if [ $? -eq 0 ]; then  
    echo "Compilation Successful" else  
    echo "Compilation Failed" fi
```

```
./26_compile.sh  
  
OMP_NUM_THREADS=4 ./omp_hello
```

```
Compilation Successful  
  
Hello from thread 3  
  
Hello from thread 2  
  
Hello from thread 1  
  
Hello from thread 0
```

27. Batch Execution

Runs the same OpenMP executable with OMP_NUM_THREADS set to 1, 2, 4, and 8, timing each run with `date +%s.%N` and `bc`. Note: this was executed on a single-core test machine, so timings look nearly identical across thread counts here — on real multi-core hardware, execution time should drop as thread count increases (up to the number of physical cores available).

```
#!/bin/bash
```

```
EXECUTABLE="./heavy_omp"
```

```
for threads in 1 2 4 8 do
```

```
    export OMP_NUM_THREADS=$threads
```

```
    echo "Running with $threads thread(s):"
```

```
    START=$(date +%s.%N)
```

```
    $EXECUTABLE
```

```
    END=$(date +%s.%N)
```

```
    ELAPSED=$(echo "$END - $START" | bc)
```

```
    echo "Execution time with $threads thread(s): $ELAPSED seconds"    echo "-----"
-----" done
```

```
./27_batch.sh
```

```
Running with 1 thread(s):
```

```
Computed sum = 599999994 using 1 threads
```

```
Execution time with 1 thread(s): 0.202614330 seconds ----- Running with 2
thread(s):
```

```
Computed sum = 599999994 using 2 threads
```

```
Execution time with 2 thread(s): 0.202341548 seconds ----- Running with 4
thread(s):
```

```
Computed sum = 599999994 using 4 threads
```

```
Execution time with 4 thread(s): 0.195760847 seconds ----- Running with 8
thread(s):
```

```
Computed sum = 599999994 using 8 threads
```

```
Execution time with 8 thread(s): 0.198485610 seconds
```

```
-----
```

28. Command Line Arguments

```
#!/bin/bash num1=$1
```

```
num2=$2

echo "Sum of $num1 and $num2 is: $((num1 + num2))"
```

```
./28_cliargs.sh 15 25
```

```
Sum of 15 and 25 is: 40
```

29. Backup Script

```
#!/bin/bash mkdir -p Backup

cp *.c Backup/ 2>/dev/null

echo "Backup completed. Files copied to Backup/:" ls Backup/
```

```
ls

./29_backup.sh
```

```
file3.c  mpi_hello.c  omp_hello.c

Backup completed. Files copied to Backup/: file3.c mpi_hello.c omp_hello.c
```

30. Mini Project: project_setup.sh

A single script that automates the entire project scaffolding: creates folders, creates placeholder source files in each, makes every .sh file executable, prints the resulting directory tree, and confirms completion.

```
#!/bin/bash
```

```
echo "Setting up Parallel Programming project structure..."
```

```
# Step 1: Create folders mkdir -p OpenMP MPI CUDA Results
```

```
# Step 2: Create placeholder source files cat > OpenMP/omp_placeholder.c << 'INNER'
```

```
// OpenMP placeholder source file
```

```
#include <stdio.h> int main() {  
    printf("OpenMP placeholder\n");    return 0;  
}
```

```
INNER
```

```
cat > MPI/mpi_placeholder.c << 'INNER'
```

```
// MPI placeholder source file
```

```
#include <stdio.h> int main() {  
    printf("MPI placeholder\n");    return 0;  
}
```

```
INNER
```

```
cat > CUDA/cuda_placeholder.cu << 'INNER'
```

```
// CUDA placeholder source file
```

```
#include <stdio.h> int main() {
```

```
    printf("CUDA placeholder\n");    return 0;
```

```
}
```

```
INNER
```

```
touch Results/results_placeholder.txt
```

```
# Step 3: Give execute permission to all .sh files find . -name "*.sh" -exec chmod +x {} \;
```

```
# Step 4: Display created directory structure echo ""
```

```
echo "Directory structure created:" find . -type d | sort echo "" echo "Files created:" find . -type f | sort
```

```
# Step 5: Completion message echo ""
```

```
echo "Project setup completed successfully."
```

```
chmod +x project_setup.sh ./project_setup.sh
```

```
Setting up Parallel Programming project structure...
```

```
Directory structure created:
```

```
.
```

```
./CUDA
```

```
./MPI
```

```
./OpenMP
```

```
./Results
```

```
Files created:
```

```
./CUDA/cuda_placeholder.cu
```

```
./MPI/mpi_placeholder.c
```

```
./OpenMP/omp_placeholder.c
```

```
./Results/results_placeholder.txt
```

```
./project_setup.sh
```

```
Project setup completed successfully.
```